

Key Dates

Handed out:	September 24, 2026
Due date:	October 8, 2026 (before 11:59 PM)

Objectives

The topic of the third homework is **graph/network visualization** in a **geospatial** context. The dataset is large, and managing the data and the visualization's visual complexity are important aspects as well.

Dataset

The data lists most (all?) worldwide commercial flights as of 10 years ago. The original data comprises information about over 7,000 airports and over 55,000 flights between these airports. For each airport, the data indicates the name, address, IATA (*International Air Transport Association*) code, and geolocation of the airport (latitude, longitude). For each flight, the data contains the origin airport IATA code, the destination airport IATA code, the airline IATA code, and the type of aircraft. The goal of this assignment is to visualize the graph formed by these flights while avoiding lag and clutter. Similar to the second assignment, you will include the necessary graphical user interface and legends in your visualization. For this assignment, I have reduced the size of the original data by merging all the flights that connect any two airports into a single summary entry. There are about 18,000 edges in the resulting data connecting about 3,400 airports.

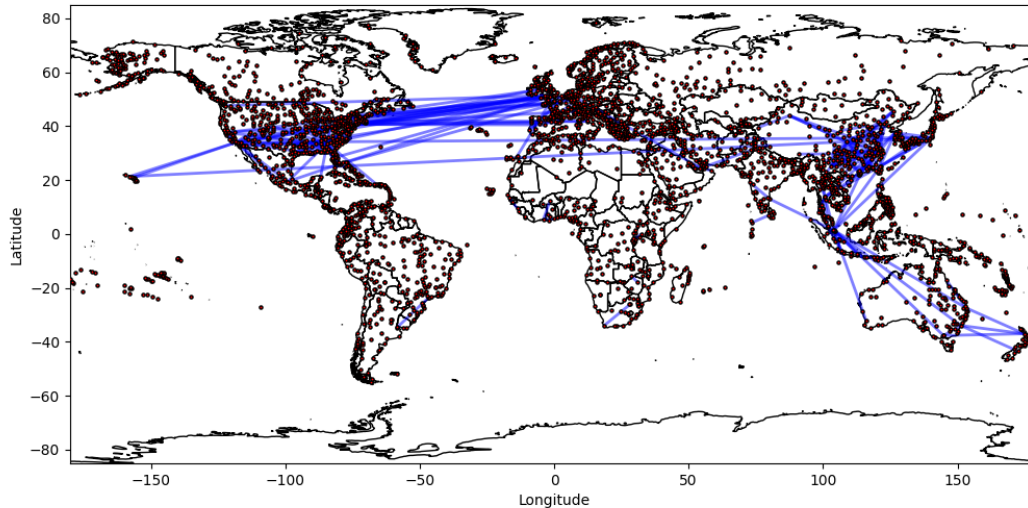
Tasks

Task 1: Geospatial Visualization (15%)

First, visualize this graph by mapping each vertex (airport) to its geospatial location and showing the graph overlaid on a map of the world. **Position each city/airport** in the dataset at its geospatial coordinates (included in the dataset) and display the corresponding node/vertex **on a map of the world** that delineates the border of each country (provided in [geojson](#) format). Finally, visualize the graph's **200 "heaviest"**

edges, where you will use the **number of flights** connecting the corresponding airports as a weight for each edge (information also contained in the dataset). Here is an illustration of possible results.

World Airports and 200 Busiest Routes



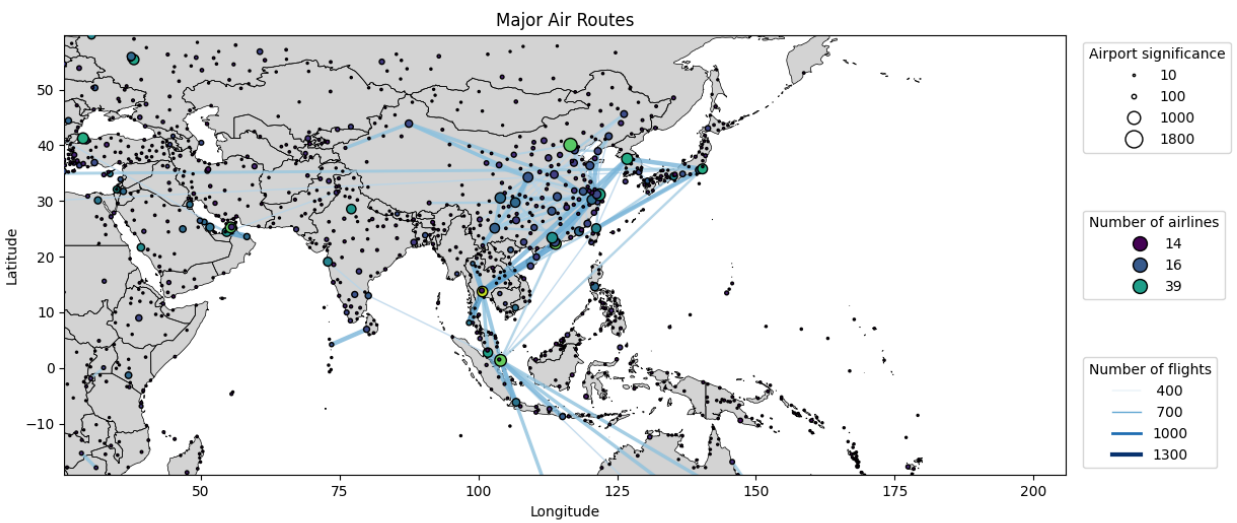
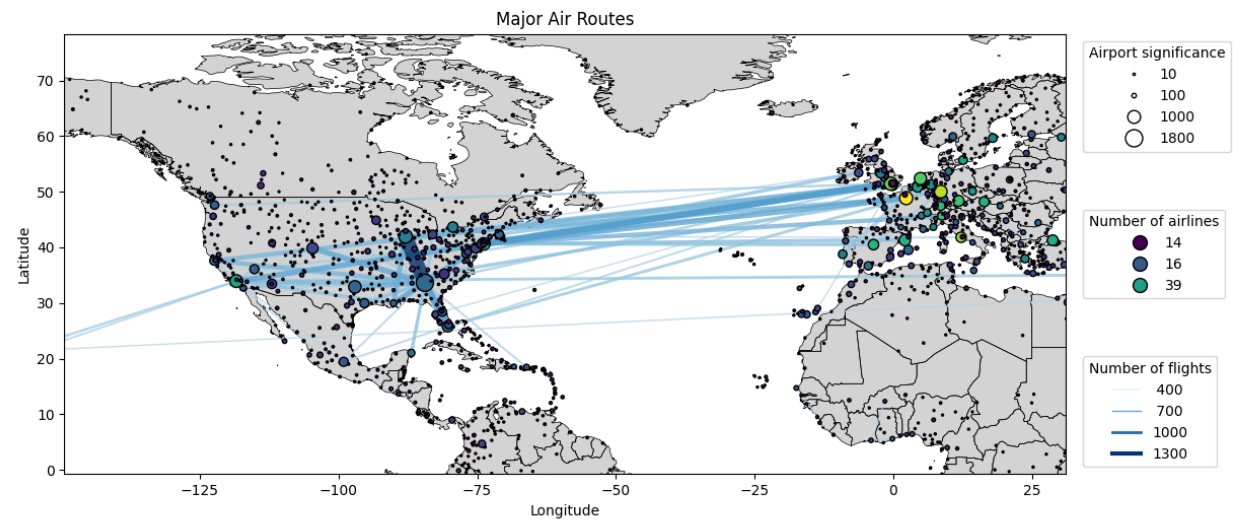
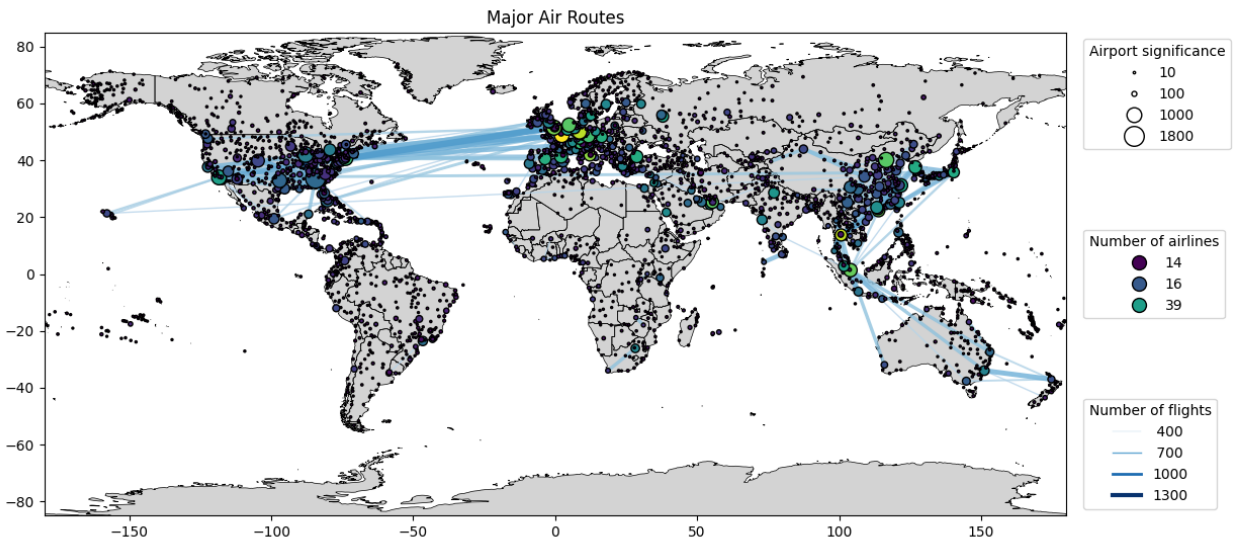
Deliverable: hw3_geospatial.py

API: python hw3_geospatial.py -m <worldmap> -a <airports> -f <flights>

Task 2: Encoding Additional Attributes (20%)

So far, the weights associated with the edges and the significance of each airport have not been visualized.

For this second task, encode the **weight of each edge** (see the definition above) using both **thickness** (or width) and **color**. In addition, compute an **importance factor for each airport** and map this value to the size (area) of the corresponding node. Here, the importance factor is the sum of the weights of all the edges incident to a given vertex, which is equivalent to the total number of flights that leave and depart from a given airport. For the color coding of the edges, vary the saturation of an edge to represent the associated weight: A higher weight should correspond to higher saturation ("purer" color). Your visualization must **include a legend** showing the meaning of node sizes (total number of flights), edge width, and color (number of connections). The nodes will be color-coded according to the number of airlines that connect to them, with an accompanying legend. Again, you will limit the number of displayed edges to the 200 heaviest ones, as in the previous task.

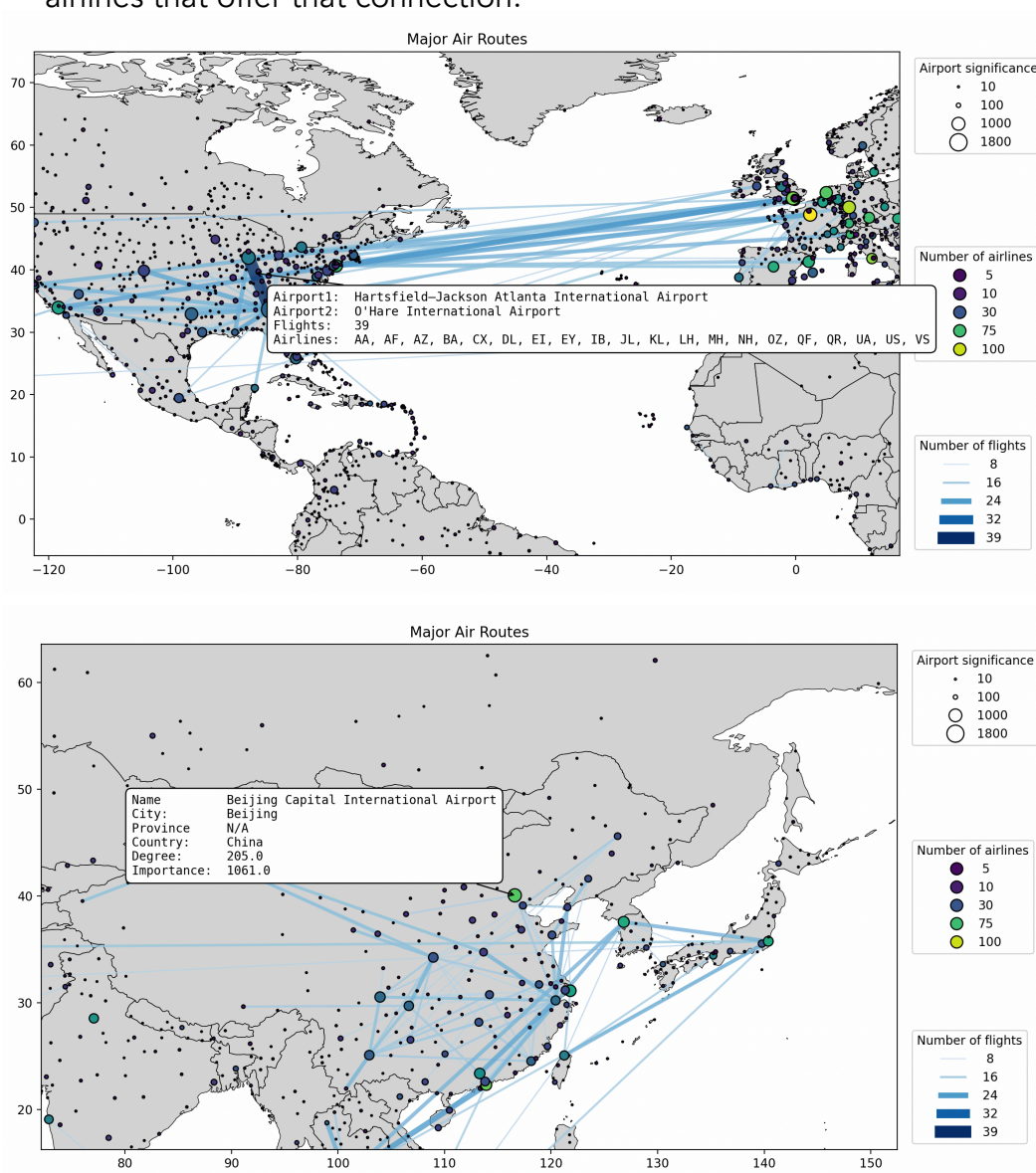


Deliverable: hw3_attributes.py

API: python hw3_attributes.py -m <worldmap> -a <airports> -f <flights>

Task 3: Tooltip (20%)

You will enhance your visualization by adding a tooltip that displays additional information about vertices and edges. For each vertex, your tooltip will indicate 1) the name of the airport/city, 2) the state or province (if available), 3) the number of other airports it is connected to (i.e., its degree), and 4) its importance factor, as defined above. For each edge, your tooltip will show 1) the names of the two airports connected by this flight, 2) the total number of flights, and 3) the names of all the airlines that offer that connection.

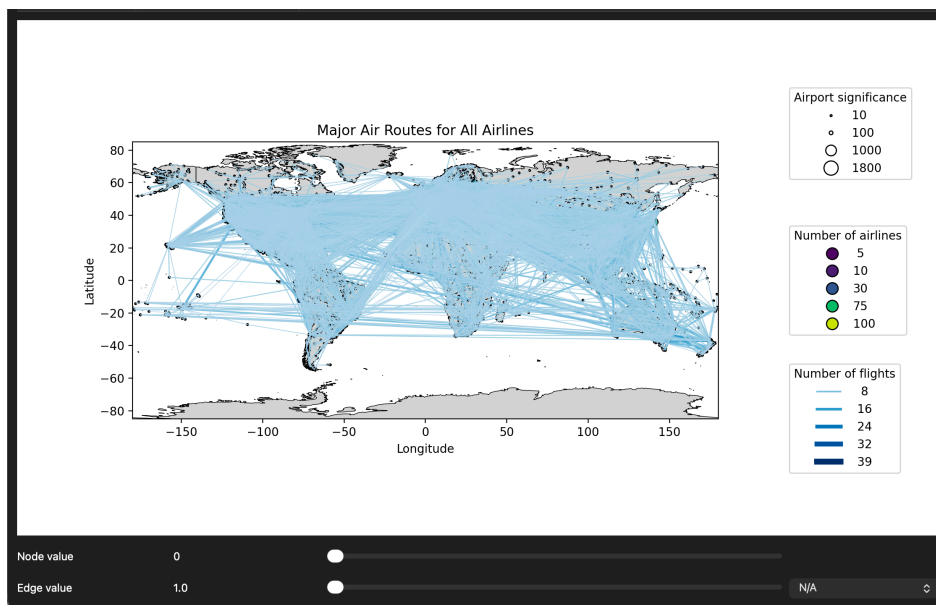


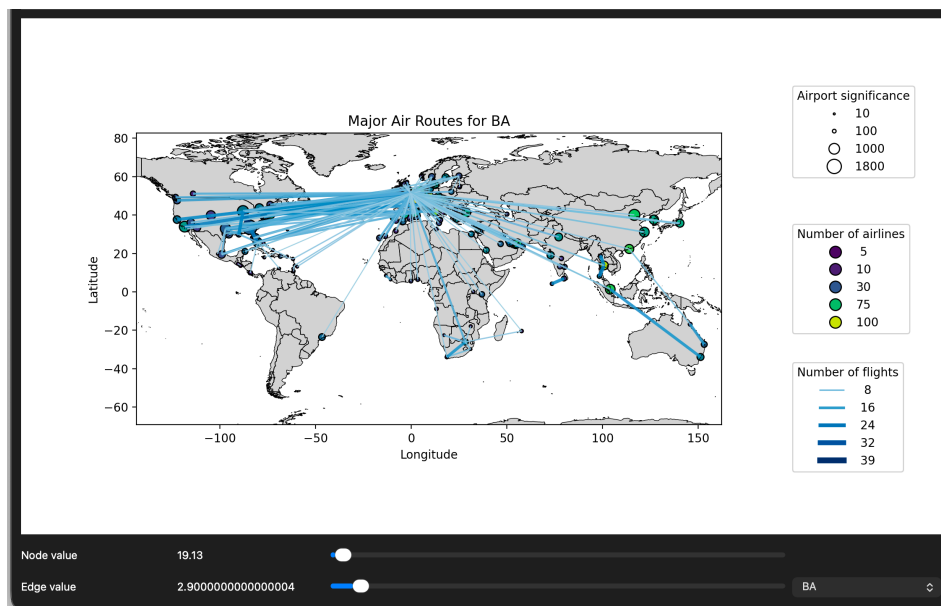
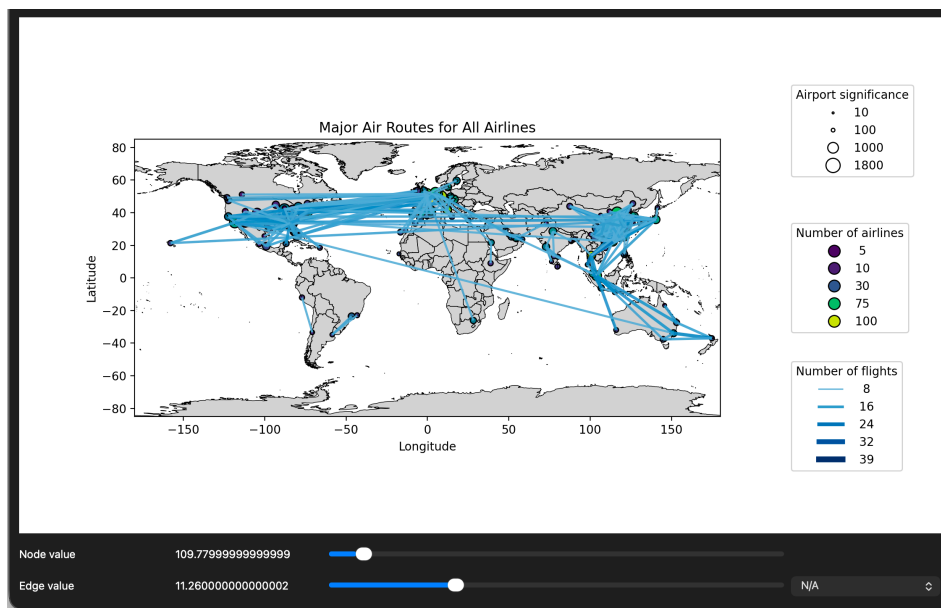
Deliverable: hw3_tooltip.py

API: python hw3_tooltip.py -m <worldmap> -a <airports> -f <flights>

Task 4: Interactive Filtering and Manipulation (25%)

You probably noticed that the number of nodes and edges makes it difficult to perceive the details of the dataset. To address this problem, you will implement three filtering mechanisms. First, create two sliders acting as filters on nodes and edges. Your **first slider** will control the **minimum importance factor for an airport** to appear in the visualization. Similarly, the **second slider** will control the **minimum weight of visible edges**. Note that applying no filter will result in over 18,000 edges! Additionally, you will implement an **airline filtering** mechanism. Specifically, your GUI should allow the user to **pick a particular airline** through a **dropdown menu** from which the code of an airline can be selected. This selection will restrict the visualization to only those flights that are operated by the selected airline. Note that **all filters must be kept consistent**: the edges associated with vertices that the first slider has removed or that are not operated by the selected airline, if any, should not be shown, regardless of their weight. Similarly, the nodes whose edges' weight is below the edge weight threshold (second slider) should not be displayed (since they will have no incident edges). Note that in contrast to previous tasks, the edge weight will be defined here as the number of flights operated by the selected airline, if any, between the corresponding two airports. The following images show examples of no filtering (first image), filtering by node importance and edge weight (2nd image), and filtering by airline 'BA' (British Airways) (third picture).





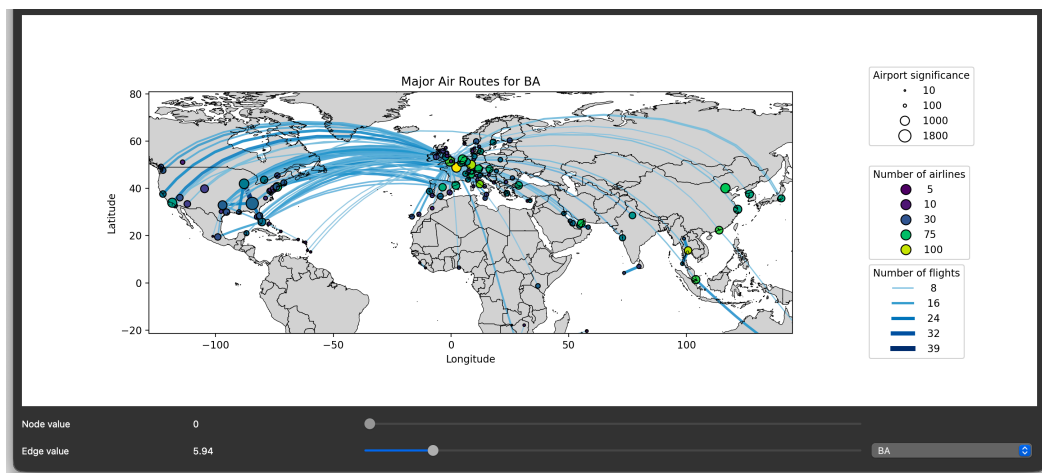
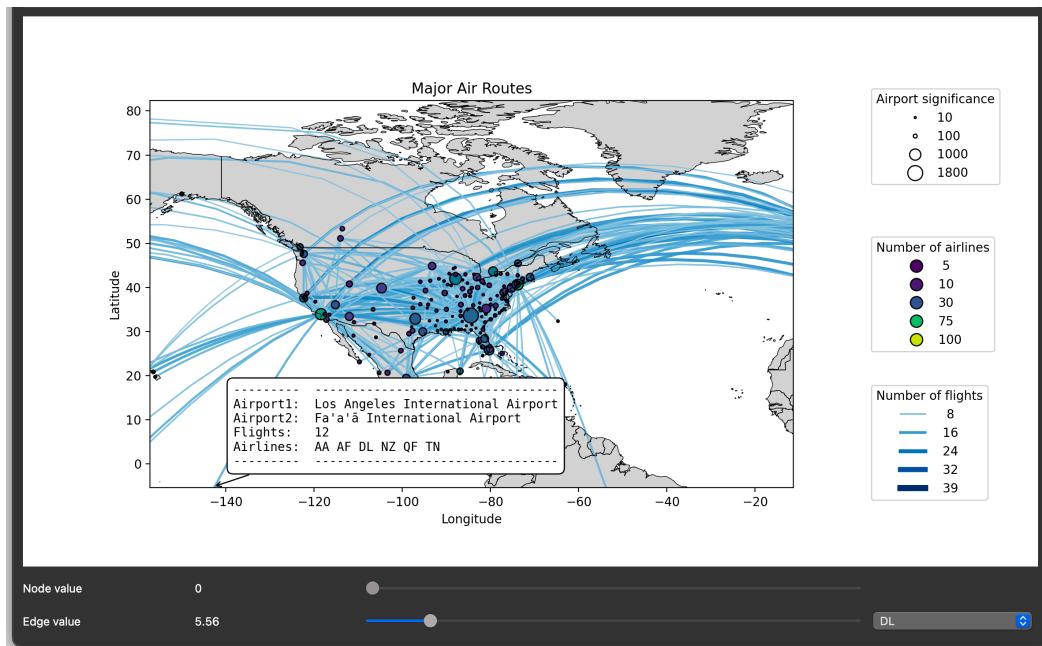
Deliverable: hw3_filtering.py

API: python hw3_filtering.py -m <worldmap> -a <airports> -f <flights>

Task 5: Geodesics (20%)

Finally, you will replace straight lines by [geodesics](#) in your representation of the flights, thereby showing the true flight path. Libraries exist to compute geodesic between two geolocations, e.g. [geographiclib](#). The following two images show selections corresponding to 'DL' (Delta Airlines) and 'BA' (British Airlines), with trajectories

visualized as geodesic paths. Note that the filtering mechanisms of the previous task should remain supported in this final task.



Deliverable: hw3_geodesic.py

API: python hw3_geodesic.py -m <worldmap> -a <airports> -f <flights>

Data Set

The dataset is comprised of three files.

[Airports \(nodes\)](#): The list of all airports and their geolocation is available in Excel format.

[Flights \(Edges\)](#): The list of all the airport connections (each connection is, in fact, the summary of all the flights that flew that route). For each connection, you will find the two airports, the corresponding number of flights, and the airlines. The file is saved in JSON format.

[World Map](#): The map of all the countries in the world. It is saved in GeoJSON format and can be easily plotted with [geopandas](#) (Matplotlib).

The data can be found on Brightspace under Content/Datasets/Homework3

Submission

Submit your solution to this assignment on Gradescope before **midnight on Thursday, October 8, 2026**.

- Include all 5 requested Python code files: hw3_geospatial.py, hw3_attributes.py, hw3_tooltip.py, hw3_filtering.py and hw3_geodesic.py.
- Include all submitted files in a single directory named <myLogin>_hw3, where <myLogin> is your Purdue login. Do not include any data file in your submission. Do not include your python environment in your submission.
- Check on the command line that your programs run and correctly **implement the requested API**.
- Check that your programs **display the visualization on the screen** (instead of saving an image in a file).